

Supplementary Material

DACT: Dynamics-Aware Learning of Coupled Torque Policies for Disturbance-Robust Legged Locomotion

I. TRAINING

A. Training Reward Structure

Reward	Equation (r_i)	Weight S1 (w_i^1)	Weight S2 (w_i^2)
Lin. velocity tracking	$\exp\{-4(\dot{\mathbf{p}}_{xy}^{ex} - \dot{\mathbf{p}}_{xy})^2\}$	1.0	1.0
Ang. velocity tracking	$\exp\{-4(\dot{\Theta}_{yaw}^{ex} - \dot{\Theta}_{yaw})^2\}$	0.5	0.5
DOF tracking reward	$\exp(-4\ \mathbf{q}^{des} - \mathbf{q}\ ^2)$	0.1	0.1
Coupled Torque Alignment	$\max(0, \cos.\text{sim}(\mathbf{a}^\tau, \tau^{fb}))$	0.40	0.40
Weighted Coupled Torque Conflict	$\frac{m_r}{m_r + \Delta m} \min(0, \cos.\text{sim}(\mathbf{a}^\tau, \tau^{fb}))$	0.10	0.10
Feedback torque magnitude	$\ \tau^{PD}\ ^2$	-2.0×10^{-5}	-2.0×10^{-5}
Feedforward torque magnitude	$\ \mathbf{a}^\tau\ ^2$	-1.0×10^{-5}	-1.0×10^{-5}
Linear velocity (z)	$\dot{\mathbf{p}}_z^2$	-2.0	-2.0
Angular velocity (xy)	$ \omega_{xy} ^2$	-0.05	-0.20
Orientations	$ \mathbf{g} ^2$	-0.2	-2.0
Joint accelerations	$\ \ddot{\mathbf{q}}\ ^2$	-2.5×10^{-7}	-2.5×10^{-7}
Joint power	$ \tau^{ex} \dot{\mathbf{q}} $	-2×10^{-5}	-2×10^{-5}
Body height	$(\mathbf{p}_z^{des} - \mathbf{p}_z)^2$	-2.0	-2.0
Action rate	$\ \mathbf{a}_t - \mathbf{a}_{t-1}\ ^2$	-0.001	-0.01
Smoothness	$(\mathbf{a}_t - 2\mathbf{a}_{t-1} + \mathbf{a}_{t-2})^2$	-0.001	-0.01
Power distribution	$\text{var}(\tau^{ex} \cdot \dot{\mathbf{q}})^2$	-10^{-5}	-10^{-5}
Joint torque limit violation	$\max(0, \tau^{ex} - \tau^{\max})^2 + \max(0, \tau^{\min} - \tau^{ex})^2$	-0.01	-1.0
Joint pose limit violation	$\max(0, \mathbf{q} - \mathbf{q}^{\max})^2 + \max(0, \mathbf{q}^{\min} - \mathbf{q})^2$	-2.0	-2.0
Feet contact force penalty	$\sum_k \max(0, \ \mathbf{f}^r\ - 200.0)$	-0.01	-0.01
Foot clearance	$(\mathbf{p}_{f,z,k}^{des} - \mathbf{p}_{f,z,k})^2 \cdot \mathbf{v}_{f,xy,k}$	-0.01	-0.01
Feet air-time reward	$\sum_k \mathbb{1}_k^{\text{first}} (0.5 - t_0) \mathbb{I}(\ \mathbf{v}_{cmd}\ > v_0)$	0.5	0.5
Foot slip penalty	$\sum_k \mathbb{I}(\mathbf{f}_{z,k}^r > \mathbf{f}_0^r) \ \mathbf{v}_{f,xy,k}\ ^2$	-0.1	-0.1
VHIP angle	$\max\left(0, \left \cos^{-1}\left(\frac{p_z}{\ell_{vh\text{hip}}}\right) - \theta_0\right \right)$	-0.1	-0.1
VHIP angular acc.	$\max\left(0, \left \frac{q}{\ell_{vh\text{hip}}} \sin(\theta_{vh\text{hip}}) - \ddot{\theta}_0\right \right)$	-0.001	-0.001
Support polygon	$\begin{cases} \exp\left(-\frac{\ \mathbf{p}_b^{xy} - \mathbf{p}_s\ - 0.6r_s\ _+^2}{\sigma}\right), & S \geq 3 \\ \exp\left(-\frac{[d(\mathbf{p}_b^{xy}, \mathbf{p}_i \mathbf{p}_j) - 0.3\ \mathbf{p}_i - \mathbf{p}_j\]_+^2}{\sigma}\right), & S = 2 \\ 0, & S < 2 \end{cases}$	0.20	0.20
Front foot overreach	$s_m \sum_{i \in \mathcal{F}} \mathbb{1}[f_i^z > 5] [x_i^b - x_{\max}]_+^2$	-100.0	-100.0
Rear foot overreach	$\sum_{i \in \mathcal{R}} \mathbb{1}[f_i^z > 5] [x_i^b - x_{\text{rear}}^{\text{nom}} - \delta_{\text{rear}}]_+^2$	-10.0	-10.0

TABLE III

REWARD TERMS, EQUATIONS, AND ASSOCIATED WEIGHTS FOR EACH TRAINING CURRICULUM STAGE. $(\cdot)^{ex}$ DENOTES THE EXPECTED COMMANDED VALUES. $\exp(\cdot)$ AND $\text{var}(\cdot)$ DENOTE THE EXPONENTIAL AND VARIANCE OPERATORS RESPECTIVELY. AN ADDITIONAL SMALL ALIVE BONUS OF 0.001 IS ADDED AT EACH TIMESTEP AND A COLLISION PENALTY OF -10.0 IS ALSO ADDED FOR EVERY LINK THAT EXPERIENCES CONTACT FORCES OUTSIDE OF THE FEET AND CALF LINKS. **S1** AND **S2** DENOTE THE WEIGHT VALUES USED IN THE FIRST AND SECOND STAGES OF THE TRAINING CURRICULUM RESPECTIVELY. THE WEIGHT VALUES WERE ANNEALED WITH A COSINE SCHEDULE FROM THE VALUES IN **S1** TO THOSE IN **S2** OVER 500 TRAINING ITERATIONS.

For fairness, all approaches explored in the main text are trained with the reward formulation in Table III. Moreover, all

approaches share the same two-stage training routine. Weights in **S1** correspond to Stage I, *Dynamics-Aware Policy Discovery*, and weights in **S2** correspond to Stage II, *Disturbance Adaptation*; S2 weights are introduced with a 500-iteration cosine schedule starting at iteration 6,000.

Most terms are adapted from [1], with additional rewards and penalties designed for coupled torque–position control. A small DOF tracking reward encourages meaningful position targets by penalizing deviations between realized joint positions and the postures induced by the coupled actions. Separate magnitude penalties are applied to feedforward torques $\mathbf{a}^\tau$ and feedback torques $\boldsymbol{\tau}^{PD}$, allowing asymmetric regularization of the two control channels. Action-rate and smoothness penalties are applied to the combined action $\mathbf{a} = \mathbf{a}^q \cup \mathbf{a}^\tau$, while joint pose and torque-limit penalties enforce kinematic and actuator feasibility. We also penalize excessive ground reaction forces and foot slip during contact.

To improve torso stability, we add VHIP-inspired penalties based on the virtual pendulum between the robot CoM and contact-force-weighted CoP [2]. The pendulum angle $\theta = \cos^{-1}(|p_{\text{CoM},z}|/\|\mathbf{p}_{\text{CoM}} - \mathbf{p}_{\text{CoP}}\|)$ is penalized above $\theta_{\text{th}} = 0.1$ rad to discourage large roll/pitch deviations, and the angular acceleration $\ddot{\theta} = g \sin(\theta)/\|\mathbf{p}_{\text{CoM}} - \mathbf{p}_{\text{CoP}}\|$ is penalized above $\ddot{\theta}_{\text{th}} = 0.01$ to encourage early correction toward stable configurations. A support-polygon reward further encourages the base projection $\mathbf{p}_b^{xy}$ to remain near the stance support set $S = \{i : f_i^z > 5\text{N}\}$: for $|S| \geq 3$, it penalizes distance beyond 0.6 of the stance radius r_S ; for $|S| = 2$, it penalizes lateral distance from the stance-foot segment beyond 0.3 of the segment length; and for $|S| < 2$, the reward is zero. The support-polygon reward σ value is $\sigma = 0.01$.

Finally, we include foot-overreach penalties to prevent stance feet from contacting the ground far from their nominal base-frame locations, which reduces the legs’ ability to generate torso-stabilizing torques under payloads. Let $[x]_+ = \max(0, x)$, let x_i^b denote the i th foot’s base-frame x position, and identify stance feet by $f_i^z > 5$ N. The front-foot penalty discourages stance front feet $\mathcal{F}$ from exceeding $x_{\text{max}} = 0.28$, with mass-dependent scaling $s_m = 1 - \frac{1}{2} \frac{m_r}{m_r + m_p}$ to relax the constraint without added payloads. The rear-foot penalty keeps stance rear feet $\mathcal{R}$ near $x_{\text{rear}}^{\text{nom}} = -0.20$ within margin $\delta_{\text{rear}} = 0.08$, penalizing both excessive forward and backward placement.

B. Domain Randomization Details

Parameter	Range S1	Range S2	Unit
Payload (Δm_{robot})	$[-1, 2]$	$[-1, 8]$	kg
Planar Disturbance Impulse Velocity ($\mathbf{F}_{\text{ext}}^{xy}$)	$[-0.5, 0.5]$	$[-1.5, 1.5]$	ms
Vertical Disturbance Impulse Velocity ($\mathbf{F}_{\text{ext}}^z$)	$[-0.1, 0]$	$[-0.5, 0.0]$	ms
Disturbance Impulse Angular Velocity ($\mathbf{M}_{\text{ext}}$)	$[-0.5, 0.5]$	$[-1.5, 1.5]$	rad/s
Linear disturbance Interval	$[5.0, 15.0]$	$[5.0, 15.0]$	s
Angular disturbance Interval	$[5.0, 15.0]$	$[5.0, 15.0]$	s
K_p factor (ΔK_p)	$[0.8, 1.2]$	$[0.8, 1.2]$	Nm/rad
K_d factor (ΔK_d)	$[0.8, 1.2]$	$[0.8, 1.2]$	Nms/rad
Motor Strength Multiplicative Factor	$[0.9, 1.1]$	$[0.9, 1.1]$	N
Center of mass shift x (ΔCoM_x)	$[-0.075, 0.075]$	$[-0.20, -0.20]$	m
Center of mass shift y (ΔCoM_y)	$[-0.075, 0.075]$	$[-0.15, -0.15]$	m
Center of mass shift z (ΔCoM_z)	$[-0.075, 0.075]$	$[0.15, 0.15]$	m
Friction coefficient	$[0.2, 1.25]$	$[0.2, 1.25]$	–
Joint armature	$[0.00, 0.015]$	$[0.00, 0.015]$	Nms/rad
Joint friction	$[0.00, 0.05]$	$[0.00, 0.20]$	Nms/rad
Joint stiffness factor	$[0.00, 0.005]$	$[0.00, 0.020]$	–
Joint damping factor	$[0.20, 0.60]$	$[0.00, 0.80]$	–

TABLE IV

DOMAIN RANDOMIZATION PARAMETERS AND RANGES. **S1** AND **S2** DENOTE THE RANGES USED IN THE FIRST AND SECOND STAGES OF THE TRAINING CURRICULUM RESPECTIVELY.

The domain randomization ranges shared across all approaches are presented in Table IV. We follow standard randomized parameters from prior legged-locomotion work [1], [3], [4], and additionally randomize vertical disturbance impulses and 3-DOF torso moments. Torso disturbances are applied as impulse wrenches: the corresponding linear or angular impulse velocity is sampled from the current range and applied at randomized temporal intervals. The fairly large values for joint-level-dynamics randomization was found empirically to be critical for successful sim2real deployment of the trained **DACT** policy.

We update randomization ranges using the performance-gated curriculum in Algorithm 2 while training all approaches explored in this work. After a warm-up period of $K_{\text{warmup}} = 2000$ iterations (corresponding to the completion of training stage **S1**) have passed, the curriculum progresses through three phases: joint-level dynamics randomization, payload/CoM randomization, and external disturbance randomization. At each iteration, an exponential moving average of the tracking reward is compared against both an absolute reward threshold and a recovery threshold defined by the 0.90 quantile of recent reward history. The randomization range advances only when performance has sufficiently recovered and at least $K_{\text{step}} = 10$ iterations have elapsed since the previous update. When these conditions are met, the active phase progress is increased by $\Delta_{\text{joint}} = 0.02$, $\Delta_{\text{mc}} = 0.01$, or $\Delta_{\text{dist}} = 0.01$, and the corresponding parameter ranges are linearly interpolated between their **S1** and **S2** bounds. This gating prevents the policy from being exposed to stronger dynamics perturbations before recovering stable tracking performance under the current randomization range. Because the threshold is based on an EMA relative to recent performance, the curriculum can still progress as tracking naturally degrades under harder disturbances.

Algorithm 2: Performance-Gated Domain Randomization Curriculum

Input : Training iteration k ; mean tracking reward $\bar{r}_k$; progress variables $p_{\text{joint}}, p_{\text{mc}}, p_{\text{dist}} \in [0, 1]$; EMA factor α ; recovery ratio η ; minimum reward r_{min} ; step interval K_{step} ; progress increments $(\Delta_{\text{joint}}, \Delta_{\text{mc}}, \Delta_{\text{dist}})$

Output: Updated domain randomization ranges

- 1 Update reward EMA: $\tilde{r}_k \leftarrow (1 - \alpha)\tilde{r}_{k-1} + \alpha\bar{r}_k$
- 2 Compute recent reference performance: $r_k^* \leftarrow \text{Quantile}_{0.90}(\{\tilde{r}_i\}_{i=k-W}^k)$
- 3 Set stepping condition: $\text{step} \leftarrow (\tilde{r}_k \geq \eta r_k^*) \wedge (\tilde{r}_k \geq r_{\text{min}}) \wedge (k - k_{\text{last}} \geq K_{\text{step}})$
- 4 **if** $k \leq K_{\text{warmup}}$ *or not* **step** **then**
- 5 | Hold current randomization ranges
- 6 **end**
- 7 **else**
- 8 | **if** *phase is joint dynamics* **then**
- 9 | | $p_{\text{joint}} \leftarrow \min(1, p_{\text{joint}} + \Delta_{\text{joint}})$
- 10 | **end**
- 11 | **else if** *phase is mass/CoM* **then**
- 12 | | $p_{\text{mc}} \leftarrow \min(1, p_{\text{mc}} + \Delta_{\text{mc}})$
- 13 | **end**
- 14 | **else if** *phase is disturbance* **then**
- 15 | | $p_{\text{dist}} \leftarrow \min(1, p_{\text{dist}} + \Delta_{\text{dist}})$
- 16 | **end**
- 17 | Advance to the next phase when the active progress reaches 1.0
- 18 | $k_{\text{last}} \leftarrow k$
- 19 **end**
- 20 Update each randomized range by linear interpolation: $\mathcal{R}_j \leftarrow \mathcal{R}_j^0 + p_{\text{joint}}(\mathcal{R}_j^1 - \mathcal{R}_j^0)$
- 21 $\mathcal{R}_{mc} \leftarrow \mathcal{R}_{mc}^0 + p_{\text{mc}}(\mathcal{R}_{mc}^1 - \mathcal{R}_{mc}^0)$
- 22 $\mathcal{R}_d \leftarrow \mathcal{R}_d^0 + p_{\text{dist}}(\mathcal{R}_d^1 - \mathcal{R}_d^0)$

Algorithm 3: Construct PINN Loss

Input: Linear velocities $\dot{\mathbf{p}}_t, \dot{\mathbf{p}}_{t+1}$; angular velocities $\dot{\boldsymbol{\Theta}}_t, \dot{\boldsymbol{\Theta}}_{t+1}$; joint velocities $\dot{\mathbf{q}}_t, \dot{\mathbf{q}}_{t+1}$; generalized coordinates $\boldsymbol{\rho}$ and velocities $\dot{\boldsymbol{\rho}}$; contact forces $\mathbf{f}_{t+1}^{r,c}$; coupled torques $\boldsymbol{\tau}_t^{ex}$; CoM shift ΔCoM ; Added Mass Δm ; time step Δt ; foot frame set $\mathcal{F}_{\text{feet}}$, Instantiated Pinocchio Robot Model $\mathcal{M}_{\text{pin}}$;

Output: **Outputs:** PINN loss $\mathcal{L}_{\text{PINN}}$

- 1 Finite-difference base accelerations: $\ddot{\mathbf{p}}_{t+1} \leftarrow \frac{\dot{\mathbf{p}}_{t+1} - \dot{\mathbf{p}}_t}{\Delta t}$, $\ddot{\boldsymbol{\Theta}}_{t+1} \leftarrow \frac{\dot{\boldsymbol{\Theta}}_{t+1} - \dot{\boldsymbol{\Theta}}_t}{\Delta t}$, $\ddot{\mathbf{q}}_{t+1} \leftarrow \frac{\dot{\mathbf{q}}_{t+1} - \dot{\mathbf{q}}_t}{\Delta t}$
- 2 Assemble generalized acceleration vector $\ddot{\boldsymbol{\rho}}_t = [\ddot{\mathbf{p}}_{t+1}, \ddot{\boldsymbol{\Theta}}_{t+1}, \ddot{\mathbf{q}}_t]^\top$
- 3 Update Pinocchio inertial model with randomized torso parameters: $\mathcal{M}_{\text{pin}} \leftarrow \text{updateInertialParams}(\mathcal{M}_{\text{pin}}, \Delta m, \Delta\text{CoM})$
- 4 Compute generalized mass matrix term $\mathbf{A} \leftarrow \text{pinocchio.CRBA}(\mathcal{M}_{\text{pin}}, \boldsymbol{\rho}_{t+1})$
- 5 Compute Coriolis and gravity terms: $\mathbf{b}, \mathbf{g} \leftarrow \text{pinocchio.RNEA}(\mathcal{M}_{\text{pin}}, \boldsymbol{\rho}_{t+1}, \dot{\boldsymbol{\rho}}_{t+1}, \mathbf{0})$
- 6 Initialize contact Jacobian list: $\mathcal{J} \leftarrow \emptyset$
- 7 **foreach** foot frame $f \in \mathcal{F}_{\text{feet}}$ **do**
- 8 Retrieve foot frame index: $i_f \leftarrow \text{getFrameId}(f)$
- 9 Compute foot Jacobian: $\mathbf{J}_f \leftarrow \text{pinocchio.computeFrameJacobian}(\boldsymbol{\rho}, i_f)$
- 10 Append to contact Jacobian set: $\mathcal{J} \leftarrow \mathcal{J} \cup \mathbf{J}_f$
- 11 Stack contact Jacobians: $\mathbf{J}_c \leftarrow \text{concat}(\mathcal{J}, \text{axis} = 0)$
- 12 Compute PINN residual loss: $\mathcal{L}_{\text{PINN}} \leftarrow \left\| \mathbf{A}\ddot{\boldsymbol{\rho}}_{t+1} + \mathbf{c}\dot{\boldsymbol{\rho}}_{t+1} + \mathbf{g} - \begin{pmatrix} \mathbf{0}_6 \\ \boldsymbol{\tau}_t^{ex} \end{pmatrix} - \mathbf{J}_c^\top \mathbf{f}_{t+1}^{r,c} \right\|^2$
- 13 **return** $\mathcal{L}_{\text{PINN}}$

The procedure used to construct the PINN losses introduced in the main text is summarized in Alg. 3. To compute the dynamics terms $\mathbf{A}$, $\mathbf{c}$, $\mathbf{g}$, and $\mathbf{J}_c$, we use the Pinocchio rigid-body dynamics library [5]. Linear and angular accelerations of the robot torso and the angular accelerations of the leg joints are approximated using a first-order backward finite difference of simulator-provided velocities. The Pinocchio robot model is then updated to account for the randomized torso CoM and payload mass. The generalized positions and velocities are passed to Pinocchio to compute whole-body dynamics terms. Specifically, Pinocchio’s composite rigid body algorithm (CRBA) is used to evaluate the generalized mass matrix $\mathbf{A}$, while the recursive Newton–Euler algorithm (RNEA) computes the combined Coriolis and gravity vector $(\mathbf{b}, \mathbf{g})$. Foot contact Jacobians are computed for each contacting foot and stacked to form the block contact Jacobian $\mathbf{J}_c$. All of the computed values are then used to calculate the whole-body dynamics loss. Since Pinocchio does not natively support batch computation, the steps in Alg. 3 are executed independently for each sample in the batch. To mitigate the resulting computational cost, we parallelize these computations as batch operations using Python multiprocessing with shared and persistent memory across CPU cores. The resulting dynamics quantities are then reshaped into batch form and used to evaluate the PINN loss in a vectorized manner. In practice, the dynamics quantities $\mathbf{A}$, $\mathbf{b}$, $\mathbf{g}$, and $\mathbf{J}_c$ are computed during the PPO simulation step and cached for subsequent loss evaluation. They are presented inline with the loss computation in Alg. 3 for clarity of exposition. Without the PINN loss, each rollout-and-update iteration required 3.16 (s) on average; while our optimized PINN implementation increased this to 4.68 (s) under identical settings on an Intel Core i9-13900KF, corresponding to (48.1%) overhead.

D. Training Algorithm

Algorithm 4: DACT PPO

Input : Encoder $\Upsilon_{\psi}^{\text{ENC}}$, privileged decoder $\Upsilon_{\delta}^{\text{DEC}}$, GRF decoder $\Upsilon_{\eta}^{\text{GRF}}$, coupled policy π_{θ}^c , value function V_{ϕ} ; environment $\mathcal{E}$; rollout horizon T ; discount γ ; PPO clip ϵ ; value, entropy, and GRF weights (c_v, c_e, λ_f)

Output : Updated parameters $(\psi, \delta, \eta, \theta, \phi)$

- 1 Initialize networks and on-policy buffer $\mathcal{D}$
- 2 **for** iteration $k = 1, 2, \dots$ **do**
- 3 **Rollout phase**
- 4 **for** $t = 1$ **to** T **do**
- 5 Observe $\mathbf{o}_t$ and privileged observation $\mathbf{o}_t^p$, including current GRFs
- 6 Update observation history $\mathbf{o}_t^H$
- 7 Encode $(\mathbf{z}_t, \hat{\mathbf{e}}_t) \sim \Upsilon_{\psi}^{\text{ENC}}(\mathbf{o}_t^H)$
- 8 Sample $(\mathbf{a}_t^q, \mathbf{a}_t^r) \sim \pi_{\theta}^c(\cdot | \mathbf{o}_t, \mathbf{z}_t, \hat{\mathbf{e}}_t)$
- 9 Execute the coupled action and observe $(r_t, \mathbf{o}_{t+1}, \mathbf{o}_{t+1}^p, d_t)$
- 10 Store transition, log probability, nominal and applied torques, measured GRFs, aligned dynamics quantities, and auxiliary-loss validity masks in $\mathcal{D}$
- 11 **end**
- 12 Compute bootstrapped returns and normalized advantages $\hat{R}_t, \hat{A}_t$
- 13 **foreach** minibatch $\mathcal{B} \subset \mathcal{D}$ **do**
- 14 **Policy and value optimization**
- 15 Freeze decoder parameters, retaining input derivatives
- 16 Encode $(\mathbf{z}_t, \hat{\mathbf{e}}_t) \sim \Upsilon_{\psi}^{\text{ENC}}(\mathbf{o}_t^H)$
- 17 Compute $\mathcal{L}_{\text{PPO}}$ using π_{θ}^c and V_{ϕ}
- 18 From the current policy mean action, compute differentiable nominal torque $\hat{\tau}_t^{\text{nom}}$ and actuator-torque estimate $\hat{\tau}_t^{\text{cmd}}$
- 19 Predict $\hat{\mathbf{f}}_{t+1}^{\pi} \leftarrow \Upsilon_{\eta}^{\text{GRF}}(\mathbf{z}_t, \hat{\mathbf{e}}_t, \hat{\tau}_t^{\text{nom}})$
- 20 Construct $\mathcal{L}_{\text{PINN}}^{\pi}$ using $\mathcal{B}$, $\hat{\tau}_t^{\text{cmd}}$, and $\hat{\mathbf{f}}_{t+1}^{\pi}$ via Algorithm 3
- 21 Update (ψ, θ) using prioritized gradient projection, with PPO primary and PINN auxiliary; update ϕ using the value loss
- 22 **Encoder-only optimization**
- 23 Freeze actor, critic, and decoder parameters
- 24 Re-encode $(\mathbf{z}_t, \hat{\mathbf{e}}_t) \sim \Upsilon_{\psi}^{\text{ENC}}(\mathbf{o}_t^H)$
- 25 Reconstruct $\hat{\mathbf{y}}_{t+1}^p \leftarrow \Upsilon_{\delta}^{\text{DEC}}(\mathbf{z}_t, \hat{\mathbf{e}}_t)$
- 26 Predict $\hat{\mathbf{f}}_{t+1}^{r,c} \leftarrow \Upsilon_{\eta}^{\text{GRF}}(\mathbf{z}_t, \hat{\mathbf{e}}_t, \tau_t^{\text{nom}})$, using detached recorded nominal torque
- 27 Compute $\mathcal{L}_{\text{PCE}}$ from privileged and GRF reconstruction, explicit-state supervision, and KL regularization
- 28 Construct $\mathcal{L}_{\text{PINN}}^{\text{ENC}}$ using $\mathcal{B}$, recorded applied torque τ_t^{app} , and $\hat{\mathbf{f}}_{t+1}^{r,c}$
- 29 Update ψ using prioritized gradient projection, with $\mathcal{L}_{\text{PCE}}$ primary and $\mathcal{L}_{\text{PINN}}^{\text{ENC}}$ auxiliary
- 30 **Decoder optimization**
- 31 Unfreeze both decoders and obtain detached outputs $(\mathbf{z}_t, \hat{\mathbf{e}}_t)$ from the frozen encoder
- 32 Recompute $\hat{\mathbf{y}}_{t+1}^p$ and $\hat{\mathbf{f}}_{t+1}^{r,c}$, conditioning the GRF decoder on recorded τ_t^{nom}
- 33 Update (δ, η) using only $\mathcal{L}_{\text{DEC}} = \|\hat{\mathbf{y}}_{t+1}^p - \mathbf{y}_{t+1}^p\|_2^2 + \lambda_f \mathcal{L}_{\text{GRF}}$
- 34 Restore encoder, actor, and critic trainability
- 35 **end**
- 36 Apply layerwise spectral normalization to $\Upsilon_{\psi}^{\text{ENC}}$ and π_{θ}^c
- 37 **end**

Policy optimization combines PPO, privileged reconstruction, and whole-body dynamics consistency objectives through alternating updates. The privileged decoder reconstructs the next-step observation excluding GRFs, $\mathbf{y}_{t+1}^p$, while a separate GRF decoder predicts contact forces conditioned on the latent context, explicit state estimate, and nominal coupled torque. Together with explicit-state supervision and KL regularization, these reconstruction objectives encourage the encoder to represent disturbance-relevant dynamics. The PINN loss in Algorithm ?? regularizes the encoder and policy toward representations and commands consistent with observed whole-body motion.

In practice, optimization proceeds in three stages. First, PPO and actor-facing PINN gradients update the policy and encoder, while the value function receives its value-loss gradient. Physics gradients propagate directly through the current policy’s commanded torque and indirectly through its torque-conditioned GRF predictions; decoder parameters remain fixed, but derivatives with respect to their inputs are retained. Second, an additional encoder-only forward pass optimizes privileged reconstruction, explicit-state estimation, KL regularization, and PINN consistency without invoking the actor. This pass conditions GRF predictions on detached recorded nominal torques and evaluates force balance using recorded applied torques, so physics gradients shape the context through the frozen GRF decoder. Physics gradients are projected against the primary PPO and reconstruction gradients in their respective stages. Finally, both decoders are trained exclusively through supervised

reconstruction using detached encoder outputs, preventing decoder weights from absorbing physics-residual errors. Observed motion and dynamics quantities remain fixed during all PINN updates.

Any potential gradient conflicts between $\mathcal{L}_{\text{PPO}}$ and $\mathcal{L}_{\text{PINN}}$ are resolved using the prioritized PCGrad [6] gradient projection technique introduced in [7], where $\mathcal{L}_{\text{PPO}}$ is the prioritized gradient. Finally, we apply layer-wise spectral normalization [8], [9] to the encoder and actor networks, with a maximum layer spectral norm of 6.0. Layer-wise spectral normalization rescales each linear layer by its largest singular value, bounding the layer’s Lipschitz constant and limiting how strongly small input changes can amplify through the network. For **DACT**, this stabilizes the encoder and coupled actor by reducing abrupt changes in the latent context, feedforward torques, and position targets, which helps mitigate overly aggressive torque responses during disturbance-rich training and physical deployment. For fairness, all evaluated approaches in this work likewise employ layer-wise spectral normalization. Algorithm 3 summarizes these stages, omitting normalization, and individual gradient calculations for compactness.

E. Hyperparameters

TABLE V
DACT HYPERPARAMETERS

Hyperparameter	Value
Num. Iterations S1	2000
Num. Iterations S2	6000
Num. Environments	4096
Steps Per Iteration	32
Epochs Per Iteration	5
Num. Mini-Batches	4
π^c , Υ_{enc} , and Υ_{dec} Learning Rate	3×10^{-4}
Critic Learning Rate	1×10^{-4}
Discount Factor γ	0.99
GAE λ	0.95
PPO Clip	0.2
Gradient Norm Clip	1.0
Entropy Coefficient Range	[0.005, 0.01]
Max Episode Length	20s
Action Function	elu
Encoder Υ_{enc} Layers	[128, 64, 19]
Decoder Υ_{dec} Layers	[64, 128]
Value Network Layers	[1024, 256, 128]
Policy π^c Layers	[512, 256, 128]
Policy Frequency	100Hz
PD-Feedback Frequency	500Hz

Algorithm 5: Adaptive Entropy Coefficient

Input : Performance metrics: linear tracking r_{lin} , angular tracking r_{ang} , terrain progress ℓ_{ter} ; thresholds $(\bar{r}_{\text{lin}}, \bar{r}_{\text{ang}}, \bar{\ell}_{\text{ter}})$; entropy bounds $[\beta_{\text{min}}, \beta_{\text{max}}]$; softmax temperature T_{ent}

Output: Entropy coefficient β

1 Compute normalized performance gaps:

$$g_{\text{lin}} = \frac{[\bar{r}_{\text{lin}} - r_{\text{lin}}]_+}{\bar{r}_{\text{lin}}}, \quad g_{\text{ang}} = \frac{[\bar{r}_{\text{ang}} - r_{\text{ang}}]_+}{\bar{r}_{\text{ang}}}, \quad g_{\text{ter}} = \frac{[\bar{\ell}_{\text{ter}} - \ell_{\text{ter}}]_+}{\bar{\ell}_{\text{ter}}}$$

2 Compute softmax weights: $\mathbf{w} \leftarrow \text{softmax}([g_{\text{lin}}, g_{\text{ang}}, g_{\text{ter}}]/T_{\text{ent}})$

3 Compute weighted gap: $g \leftarrow \mathbf{w}^T [g_{\text{lin}}, g_{\text{ang}}, g_{\text{ter}}]$

4 Update entropy coefficient: $\beta \leftarrow \beta_{\text{min}} + g(\beta_{\text{max}} - \beta_{\text{min}})$

5 **return** β

Table V summarizes the training hyperparameters. Base velocity commands are sampled uniformly from $[-1.0, 1.0]$ across all DOFs. During early training, the linear velocity range is restricted to $[-0.5, 0.5]$ m/s and expanded once $\pi^c(\cdot)$ achieves an average episodic linear tracking reward of 0.8. The context encoder $\Upsilon_{\text{enc}}(\cdot)$ processes a history of $H = 20$ timesteps, effectively maintaining a history of 0.20 seconds. The training terrains consist of 10×20 subterrains of size 8×8 m and a 4 m central platform. Terrain types are sampled as smooth slopes, rough slopes, up/down stairs, discrete obstacles, and wave terrain with proportions $[0.10, 0.15, 0.25, 0.25, 0.20, 0.05]$. Difficulty scales each terrain level, increasing slopes up to 0.4, roughness from 0.02–0.12 m, stair and obstacle heights from 0.04–0.20 m, and wave amplitude up to 0.20 m.

Additionally, for all approaches, we use the adaptive entropy schedule in Algorithm 5 to maintain exploration when tracking or terrain progression degrades, as proposed in [10]. The entropy coefficient is initialized to 0.01 and constrained to $[0.005, 0.01]$. At each update, we compute normalized gaps between the current performance metrics and target thresholds for linear velocity tracking (0.75), angular velocity tracking (0.35), and average terrain curriculum level (6.0). A temperature-scaled softmax with $T_{\text{ent}} = 2.0$ weights these gaps, producing a scalar performance deficit used to interpolate the entropy coefficient between its lower and upper bounds. Thus, entropy decreases as the policy achieves stable tracking and terrain progress, but increases when performance falls below the specified targets.

II. ADDITIONAL EXPERIMENTAL RESULTS

A. PINN Loss Training Performance

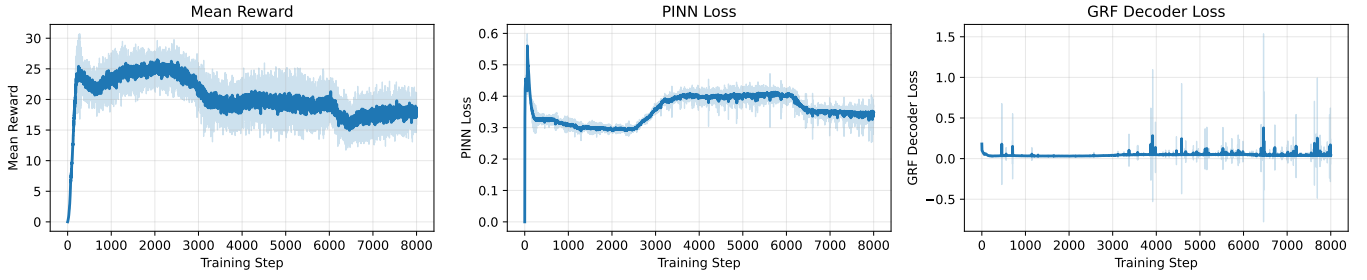


Fig. 7. **DACT** mean episodic reward and $\mathcal{L}_{\text{PINN}}$ loss performance averaged (with two standard deviations) across three random seeds.

Figure 7 shows rapid initial reward growth across three random seeds, followed by reductions as the curriculum becomes more demanding and partial recovery near the end of training. The PINN loss initially spikes, decreases to approximately 0.30, and rises to a plateau near 0.40 during the intermediate curriculum phases before settling around 0.35. This late reduction is consistent with improved whole-body dynamics consistency relative to the preceding phase. The GRF reconstruction loss decreases rapidly and remains low on average, although intermittent spikes and greater variability emerge later in training. Together, these trends suggest continued adaptation to increasingly challenging dynamics, while the transient GRF errors highlight remaining difficulty in predicting some contact responses.

III. LIMITATIONS



Fig. 8. **DACT** failure case when carrying 6.5 L liquid payload while walking up a stair case with 12 cm high steps.

Figure 8 shows a failure case in which **DACT** attempts to climb 12 cm stairs while carrying a 6.5 L liquid payload. During ascent, the front-right foot contacts and becomes caught on the inward bevel of the vertical stair face. The policy attempts to pull the foot over the stair edge, but the resulting torque demand triggers the Go1’s built-in safety cutoff, disabling the motors. We attribute this failure to two limitations of the current formulation: **DACT** is a blind policy and therefore cannot proactively adjust swing-foot placement around sharp terrain discontinuities, and its relatively stiff torque response can produce excessive corrective torques when the foot becomes constrained. Future work will investigate perception-aware foot placement and further sim2real refinement of feedforward torque generation to improve recovery from foot entrapment on discontinuous terrain.

REFERENCES

- [1] I. M. A. Nahrendra, B. Yu, and H. Myung, “Dreamwaq: Learning robust quadrupedal locomotion with implicit terrain imagination via deep reinforcement learning,” in *2023 IEEE International Conference on Robotics and Automation, ICRA 2023*. Institute of Electrical and Electronics Engineers Inc., 2023, pp. 5078–5084.
- [2] E. Xiao, Y. Dong, J. Ma, and P. Lu, “Stable imitation of multigait and bipedal motions for quadrupedal robots over uneven terrains,” *Advanced Robotics Research*, p. 202500036, 2025.
- [3] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Conference on robot learning*. PMLR, 2022, pp. 91–100.
- [4] S. Chen, B. Zhang, M. W. Mueller, A. Rai, and K. Sreenath, “Learning torque control for quadrupedal locomotion,” in *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*. IEEE, 2023, pp. 1–8.
- [5] J. Carpentier and N. Mansard, “Analytical derivatives of rigid body dynamics algorithms,” in *Robotics: Science and systems (RSS 2018)*, 2018.
- [6] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn, “Gradient surgery for multi-task learning,” *Advances in neural information processing systems*, vol. 33, pp. 5824–5836, 2020.
- [7] S. Peri, A. Perincherry, B. Pandit, and S. Lee, “Non-conflicting energy minimization in reinforcement learning based robot control,” in *9th Annual Conference on Robot Learning*.
- [8] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural-fly enables rapid learning for agile flight in strong winds,” *Science Robotics*, vol. 7, no. 66, 2022.
- [9] J. Shin, W. Cha, D. Kim, J. Cha, and J. Park, “Spectral normalization for lipschitz-constrained policies on learning humanoid locomotion,” *arXiv preprint arXiv:2504.08246*, 2025.
- [10] J. Chen, L. Kong, Z. Zhang, and Z. Xue, “Robust quadrupedal locomotion on complex terrains via adaptive entropy learning,” *Research Square Preprint* [<https://doi.org/10.21203/rs.3.rs-9269338/v1>], 2026.